

Lesotho Farmers Portal — API Migration Guide

For pilot-data migration developers

MAFSN — Ministry of Agriculture, Food Security and Nutrition

2026-05-06

Table of Contents

Lesotho Farmers Portal — API Migration Guide

Audience: Developers migrating pilot farmer + parcel data into the Lesotho Farmers Portal. **Version:** 2026-05-06. **Scope:** Farmer registry, Parcel registry, Master Data lookups (MDM).

This guide explains how to use the portal's REST API to bulk-load pilot-collected farmer and parcel records into the system. It covers authentication, the relevant endpoints, the field shapes each form expects, the master-data lookup tables, the recommended migration sequence, and a working Python migration script you can adapt.

Section 1 — Read this first

What you will be doing

You'll be calling REST endpoints that go through Joget's standard form-data save path (which means: validators run, foreign-key checks run, the audit trail is correct). Each endpoint corresponds to one form in the portal — Farmer Basic Information, Farmer Residency, Parcel Location, Parcel Geometry, etc.

Your migration runs as a sequence of POST calls — one per record per form. The portal accepts each call independently, returns the saved row's UUID, and handles all the cascade (Hibernate mapping, storeBinder logic, etc.) on its side.

What you must NOT do

- **Do not write directly to the Postgres `app_fd_*` tables.** Joget caches form definitions and Hibernate mappings in JVM memory, and a raw INSERT silently bypasses both. See the portal's CLAUDE.md "Hard rule" section for why this corrupts the system in subtle ways. Always use the API.
- **Do not skip validation.** If a record violates a mandatory or uniqueness check, the API returns HTTP 400 with the error. Don't retry with bypassed validation — fix the record.

- **Do not create master-data rows your migration introduces.** If your pilot data references a district, village, agro-zone, or any other master-data entity that doesn't already exist in the portal's MDM, the migration should HALT for human review. Adding rows to MDM is a separate authorisation; the migration tool isn't authorised to expand the master-data set.

What's the data model

Two registry trees:

farmerRegistrationForm (wizard parent)

- └─ farmerBasicInfo (national_id is the business key)
- └─ farmerResidency (district, agro-zone, GPS, land sizes)
- └─ farmerHousehold (with householdMemberForm child grid)
- └─ farmerCropsLivestock (with livestockDetailsForm child grid)
- └─ farmerIncomePrograms
- └─ farmerDeclaration

parcelRegistration (parcel wizard parent)

- └─ parcelLocation (district, agro-zone, GPS)
- └─ parcelGeometry (polygon, area, perimeter)
- └─ parcelClassification (land use, conservation practices)
- └─ parcelCrops (with cropDetailForm child grid)

Both trees use a parent_id HiddenField on each child to link to its wizard parent's UUID. For pilot migration you may **skip** authoring the wizard parent and just create the sub-form records — the data is queryable that way, and the wizard re-binds correctly when an operator later opens it in the UI.

The link between farmer and parcel is by the farmer's national_id: each parcelLocation record carries district + village plus the parcel's identifiers, and the parcel's link to a specific farmer is recorded in the parcel-registration tree's parent_id chain (or, in pilot data, you'll typically have an explicit "farmer_nid" column on each parcel that you map to the right field).

Section 2 — Authentication

Every API call needs **two HTTP headers**:

Header	Value	Notes
api_id	The API definition's UUID (e.g. API-a7735b09-36be-453d-a385-cdff0c3df7b0)	One per endpoint — see Section 4 for the full list
api_key	a5af1181f77b4a62b481725b6410e965	Currently the shared farmerPortal key for dev. For your migration, request a fresh dedicated

**credential pair from
MAFSN ICT** so your activity
is auditable separately.

A third optional header is Content-Type: application/json for POST bodies.

The api_id can also be passed as a query parameter (?api_id=...&api_key=...) but headers are recommended because they don't get logged in the access log.

Verifying auth works

The simplest auth-only check is to POST a deliberately-empty record and observe that the server gets to validation. The response will be HTTP 200 with a body like this (proving the request reached validation, which means auth passed):

```
curl -X POST -H "Content-Type: application/json" \
  -H "api_id: API-a7735b09-36be-453d-a385-cdff0c3df7b0" \
  -H "api_key: a5af1181f77b4a62b481725b6410e965" \
  -d '{} ' \
  "http://20.87.213.78:8080/jw/api/form/farmerBasicInfo"
```

Expected response (HTTP 200):

```
{ "id": "", "errors": { "national_id": "Missing required value",
                        "last_name": "Missing required value",
                        "gender": "Missing required value",
                        "first_name": "Missing required value" } }
```

That proves the API auth works (the validator ran). If you get HTTP 400 with "Bad Request" instead, your headers are wrong (the request got rejected before validation could run). If you get HTTP 404, the URL path is wrong (most common: missing /jw prefix or wrong host).

(We don't use a GET /list for this check because — see Section 3 — that endpoint isn't functional.)

Section 3 — URL pattern

Every per-form API has the same operation set:

POST	/jw/api/form/{formId}	Create a new record	□ works
GET	/jw/api/form/{formId}/{recordId}	Read by UUID	□ works
PUT	/jw/api/form/{formId}	Update (body must include id)	□ works
DELETE	/jw/api/form/{formId}/{recordId}	Delete by UUID	□ works
POST	/jw/api/form/{formId}/saveOrUpdate	Upsert by id	□ works
POST	/jw/api/form/{formId}/addWithFiles	Create with attached files	□ works
POST	/jw/api/form/{formId}/updateWithFiles	Update with attached files	□ works
GET	/jw/api/form/{formId}/list	(LIST)	△ NOT IMPLEMENTED

Replace {formId} with the formDefId from Section 4 (e.g. farmerBasicInfo, parcelGeometry, md03district).

/list does not work — the path is enabled in API Builder but the underlying AppFormAPI plugin doesn't implement listing; the endpoint returns an empty object. For bulk reads of master data, use the static CSVs in `mdm_reference/` (Section 6). For reading individual records you already know the UUID of, use GET `/recordId`.

The host in your environment will be different from the dev URL above — ask MAFSN ICT for the production / UAT URL.

Section 4 — Endpoint reference

4.1 Farmer registry

Form ID	API ID	Purpose
farmerBasicInfo	API-a7735b09-36be-453d-a385-cdff0c3df7b0	National ID, name, gender, DOB, contact, marital status, language, cooperative
farmerResidency	API-30dcfc48-44bf-4aaf-95df-8370590be7b4	District, agro-zone, village, GPS, land sizes, distance to services
farmerHousehold	API-d67facef-8a05-42e2-a0ab-3eb32ede0023	Household-level container (parent for member grid)
householdMemberForm	API-a24e8b71-de05-4035-bbf1-ae0be68aa5be	Per-member rows under farmerHousehold (FormGrid child)
farmerCropsLivestock	API-73375fcb-f3ed-4614-8450-72e9f0029ec1	Has-livestock indicator + livestock detail child grid
farmerIncomePrograms	API-a8a4405a-0d34-4a3b-b5c9-6bd8d6a464b1	Income source(s), employment, support programmes, loans
farmerDeclaration	API-b8c3c520-c2df-4a28-b77a-c7990f6d4295	Consent, declaration date, beneficiary code, registration channel
farmerRegistrationForm	API-068737d6-dfe3-423d-8ab7-e4299189b589	Wizard parent (optional for migration)

4.2 Parcel registry

Form ID	API ID	Purpose
parcelRegistration	API-f276b1a6-3a94-4953-9257-f1fda2175114	Parcel wizard parent
parcelLocation	API-939995e0-7cc6-4d82-ae34-fa58546f3b77	District, agro-zone, village, parcel name
parcelGeometry	API-87d12ed1-074a-4368-9d9f-169dd2f04db1	Polygon, area_hectares, perimeter_meters, centroid (set by GIS widget)

parcelClassification	API-19867883-615c-4012-9a03-d3cb22e6290e	Cultivated/conservation hectares, conservation practices
parcelCrops	API-cccdca6c-3141-4484-b24c-669399968add	Crop list (parent for cropDetailForm child grid)

4.3 Master data (MDM)

The portal has 50+ master-data forms. The ones most relevant to farmer/parcel migration are listed below. To **read** any of them during your migration, use the dedicated MDM list endpoint (GET /jw/api/regbb/mdm/list?formId=<form>) — it works for any form ID in this table that starts with md. See Section 6 for details.

Form ID	API ID	Used by
md01maritalStatus	API-b63f36c8-5fd3-4962-8a60-02dd9571735c	farmerBasicInfo.marital_status
md02language	API-5a747ccc-2c1b-4eb8-9d6c-081f9960690b	farmerBasicInfo.preferred_language
md03district	API-d793713e-5b0d-4a51-b1d0-4f92bfc76022	farmerResidency.district, parcelLocation.district
md04agroEcologicalZo	API-90b02a2c-8bbc-4a35-9b4e-dd3985e5100c	residency / parcel agro-ecological zone
md05residencyType	API-ea109d46-4992-4679-b47e-409dbcf44538	farmerResidency.residency_type
md06farmLabourSource	API-d9c5626d-c7ca-4b37-9320-42e03b699157	(informs labour reporting)
md07livelihoodSource	API-87e9df36-e3a9-4bff-86a7-d8fae9063c60	farmer income reporting
md08educationLevel	API-1392ad15-d3b6-4dfe-95d8-dc227c54e8a1	household member education
md10conservationPrac	API-56d73871-9cb6-4445-a20d-5e6b5a0e591d	parcelClassification.conserva tionPractices
md11hazard	API-6ab1f5e8-4087-4ecc-a04e-28429932c356	hazard exposure
md12relationship	API-c42f149f-76f4-4a54-bc51-48b1aa72dc79	householdMemberForm.rel ationship
md13orphanhoodStatus	API-e4793a36-50ec-4e99-8aae-7f692baf0870	householdMemberForm.orp hanhoodStatus
md14disabilityStatus	API-67127c98-e6e6-4e50-8147-38b6ed9d5d05	householdMemberForm.dis ability

md15areaUnits	API-12aad22b-1588-4b44-ab54-97dfcd355573	unit conversion
md16livestockType	API-1d668daa-66a0-41c7-b617-7a8154fa4302	livestockDetailsForm.livestockType
md161livestockCategory	API-feefac22-621f-4d41-a2e4-3dff65ebced0	livestock taxonomy
md17incomeSource	API-d84be52d-dc8c-4ca7-8d22-5d289817bd11	farmerIncomePrograms.income_sources
md18registrationChan	API-98c60d22-498b-4941-a071-614dec827817	farmerDeclaration.registrationChannel
md19crops	API-27ec6a6f-e38f-409e-ad78-4a726f5dbda6	crops the farmer grows
md191cropCategory	API-ff817174-4430-46d1-8916-6afd3cb88da3	crop taxonomy
md37collectionPoint	API-0f99b01d-60c2-4ac8-94b7-4d97827ed6bd	farmerResidency.resource_center, parcelLocation.resource_center
md42TargetCategory	API-545b64e8-3e8b-4ad7-9b82-2d6524b518bf	farmer category (smallholder, emerging, ...)

(For the full 50+ master-data list, query the /jw/api/formcreator/formcreator/apis endpoint or ask MAFSN ICT.)

Section 5 — Field reference per form

5.1 farmerBasicInfo

The registry's primary record. national_id is the business key — duplicates are blocked by the validator.

Field

national_id

first_name

last_name

gender

date_of_birth

marital_status

preferred_language

mobile_number

email_address

extension_officer_name

member_of_cooperative

cooperative_name

Example body:

```
{
  "national_id": "066257236627",
  "first_name": "Mants'ali",
  "last_name": "Panyane",
  "gender": "female",
  "date_of_birth": "1985-10-01",
  "marital_status": "married",
  "preferred_language": "english",
  "mobile_number": "+26659518328",
  "member_of_cooperative": "yes",
  "cooperative_name": "Mabote Block Farming Cooperative"
}
```

5.2 farmerResidency

Where the farmer lives + what they farm. References district, agro-zone, and resource centre by code.

Field

district

agroEcologicalZone

resource_center

village

residency_type

yearsInArea

gpsLatitude

gpsLongitude

ownedRentedLand

totalAvailableLand

cultivatedLand

conservationAgricultureLand

parent_id

(Plus distance-to-services fields: distanceWaterSource, distancePrimarySchool, distanceLivestockMarket, etc.)

5.3 farmerHousehold + householdMemberForm

farmerHousehold is just a container for household members. The actual member rows go in householdMemberForm, each with a farmer_id foreign key pointing back to the farmerHousehold's UUID.

householdMemberForm fields:

Field

farmer_id

memberName

sex

date_of_birth

relationship

orphanhoodStatus

participatesInAgriculture

disability

chronicallyIll

5.4 farmerCropsLivestock + livestockDetailsForm

farmerCropsLivestock carries one Boolean (hasLivestock). If yes, add per-livestock-type rows via livestockDetailsForm:

Field

farmer_id

livestockType

numberOfMale

numberOfFemale

5.5 farmerIncomePrograms

Field

mainSourceIncome

averageAnnualIncome

monthlyExpenditure

gainfulEmployment

governmentEmployed

relativeSupport

everOnISP

totalLoans12Months

(...)

5.6 farmerDeclaration

Field

declarationConsent
declarationFullName
field13
registrationStation
registrationChannel
beneficiaryCode

5.7 parcelRegistration + sub-forms

Parcel records mirror the farmer pattern. Most data lives in parcelLocation and parcelGeometry.

parcelLocation:

Field

parent_id
parcel_number
district
agroEcologicalZone
resource_centre
village

parcelName

parcelGeometry:

Field	Type	Notes
parent_id	UUID	parcelRegistration's id
area_hectares	numeric	Computed by GIS widget if drawn; supply directly for migration
perimeter_meters	numeric	
centroid_lat	numeric	
vertex_count	integer	Polygon centroid latitude
auto_center_lat /	numeric	Default Lesotho coords if no

auto_center_lon

polygon (use -29.6 / 28.2)

For a polygon, the actual GIS data goes into a separate `gisPolygonCapture` widget — for migration you'll typically populate just the derived `area_hectares` + the centroid.

parcelClassification:

Field

parent_id

cultivatedLand

conservationAgricultureLand

conservationPractices

conservationNote

parcelNotes

Section 6 — Migration order

The right order matters because of the FK references.

Step 1 — Inventory the master-data your pilot uses

For each MDM-referenced field in your pilot data, verify every value maps to an existing code in the portal's master data.

The portal exposes a dedicated MDM-list endpoint (build-109+):

```
GET /jw/api/regbb/mdm/list?formId=<form>
api_id: API-168e3678-1f9a-46fc-8c19-d0d9a917eb73
api_key: <your dedicated key>
```

Example:

```
curl -H "api_id: API-168e3678-1f9a-46fc-8c19-d0d9a917eb73" \
-H "api_key: <KEY>" \
"http://<host>/jw/api/regbb/mdm/list?formId=md03district"
```

Response:

```
{
  "status": "ok",
  "formId": "md03district",
  "tableName": "app_fd_md03district",
  "rowCount": 10,
  "rows": [
    {"id": "abc...", "code": "berea", "name": "Berea"},
    {"id": "def...", "code": "butha-buthe", "name": "Butha-Buthe"},
    ...
  ]
}
```

```
]
}
```

Pass any formId that starts with md (the endpoint is whitelisted to master-data forms only — it won't return application or transactional data). The c_ column prefix is stripped in the output for clean client-side handling.

Why this dedicated endpoint? Joget's standard per-form /list endpoint (e.g. /jw/api/form/md03district/list) is enabled in the API config but the underlying plugin (AppFormAPI) doesn't implement listing — calls return {}. The dedicated /regbb/mdm/list endpoint fills that gap specifically for migration / integration use cases.

Offline fallback: static CSVs. A pre-exported snapshot of all relevant master-data forms ships in mdm_reference/ next to this guide, in case you need to validate offline (e.g. while developing your migration script before the portal is reachable).

```
mdm_reference/
  districts.csv          (10 rows)
  agro_zones.csv        (4 rows)
  resource_centres.csv   (70 rows — code, name, district_code)
  marital_status.csv
  languages.csv
  residency_types.csv
  relationships.csv
  livestock_types.csv
  income_sources.csv
  conservation_practices.csv
  ... (15 files total)
```

Open these CSVs as your authoritative reference. For example, valid districts:

```
code,name
berea,Berea
butha-buthe,Butha-Buthe
leribe,Leribe
mafeteng,Mafeteng
maseru,Maseru
mohales-hoek,Mohale's Hoek
mokhotlong,Mokhotlong
qachas-nek,Qacha's Nek
quthing,Quthing
thaba-tseka,Thaba-Tseka
```

Note the convention: lower-case code, hyphen-separated. Maseru (capital M) won't match. thaba_tseka (underscore) won't match. thaba-tseka (hyphen, lowercase) is correct.

For each pilot data column that maps to an MDM code, build a translation table:

pilot_value

Maseru

Maseru District

Mafetenq

Thaba_Tseka

If your pilot has values that don't map even after normalisation (e.g. a centre that doesn't exist in `resource_centres.csv`), **stop and ask MAFSN ICT** — the master data should be authored before migration runs, not during. The migration tool isn't authorised to expand MDM.

Refresh cadence. Whenever MAFSN updates master data, the CSVs in `mdm_reference/` go stale. If your migration is more than ~2 weeks old relative to the CSV snapshot date, ask MAFSN ICT for a fresh export (they can re-run the SQL `SELECT c_code, c_name FROM app_fd_md*` for each form and post the new CSV).

The included `sample_migration.py` script reads these CSVs as its master-data source — see Section 7 for how it uses them.

Step 2 — Create farmer records

Order:

1. `farmerBasicInfo` — gets you a UUID per farmer (call this `FBI_UUID`)
2. `farmerResidency` with `parent_id = FBI_UUID` (or skip `parent_id`)
3. `farmerHousehold` if you have household data
4. `householdMemberForm` rows (one per household member, with `farmer_id` pointing at the `farmerHousehold`'s UUID)
5. `farmerCropsLivestock` + `livestockDetailsForm` rows (same pattern)
6. `farmerIncomePrograms`
7. `farmerDeclaration`

You can stop at step 1 (just `farmerBasicInfo`) for a minimal migration — the rest are optional sub-forms. The system will tolerate gaps.

Step 3 — Create parcel records

1. `parcelRegistration` — gets you a UUID (`PR_UUID`)
2. `parcelLocation` with `parent_id = PR_UUID`
3. `parcelGeometry` with `parent_id = PR_UUID`
4. `parcelClassification` with `parent_id = PR_UUID`

To link the parcel to its farmer, your pilot data should carry the farmer's NID or some other identifier on each parcel. Map that to the `farmerBasicInfo` UUID and persist it on the parcel via the `parcelLocation.parent_id` (if you choose to use the wizard parent chain) or via a custom mapping table you maintain.

Step 4 — Verify

Spot-check 5–10 records by reading them back through the API and comparing to the pilot source.

Read by NID — there's no native get-by-NID, so use the list endpoint:

```
curl -H "api_id: API-a7735b09-..." -H "api_key: ..." \
  "http://<host>/jw/api/form/farmerBasicInfo/list?national_id=066257236627"
```

Step 5 — Validate aggregate counts

Compare the count of rows in your source vs the count returned by the portal's APIs. If they don't match, find the missing records (they likely failed validation; check the migration log).

Section 7 — Sample Python migration script

A minimal end-to-end migration script lives at `_07_Implementation_Guides/sample_migration.py`. The structure:

```
import csv, json, urllib.request

BASE = "http://20.87.213.78:8080/jw" # or your environment's URL
KEY = "a5af1181f77b4a62b481725b6410e965" # ASK FOR YOUR DEDICATED KEY

API_FBI = "API-a7735b09-36be-453d-a385-cdff0c3df7b0"
API_RES = "API-30dcfc48-44bf-4aaf-95df-8370590be7b4"
API_DISTR = "API-d793713e-5b0d-4a51-b1d0-4f92bfc76022"

def post(form_id, body, api_id):
    url = f"{BASE}/api/form/{form_id}"
    req = urllib.request.Request(url, method="POST",
        data=json.dumps(body).encode(),
        headers={"Content-Type": "application/json",
            "api_id": api_id, "api_key": KEY})
    with urllib.request.urlopen(req, timeout=60) as r:
        return json.loads(r.read())

def get_list(form_id, api_id):
    url = f"{BASE}/api/form/{form_id}/list"
    req = urllib.request.Request(url,
        headers={"api_id": api_id, "api_key": KEY})
    with urllib.request.urlopen(req, timeout=60) as r:
        return json.loads(r.read())

# 1. Read MDM lookups so we can validate codes
districts = {d["code"]: d["name"] for d in get_list("md03district", API_DISTR)}
```

2. Walk the pilot CSV

```

with open("pilot_farmers.csv") as f:
    for row in csv.DictReader(f):
        # Validate district maps to an existing code
        if row["district"] not in districts:
            print(f"SKIP {row['national_id']}: unknown district {row['district']}")
            continue

        # Create the basic info record
        body = {
            "national_id": row["national_id"],
            "first_name": row["first_name"],
            "last_name": row["last_name"],
            "gender": row["gender"].lower(),
            "date_of_birth": row.get("dob", ""),
            "mobile_number": row.get("phone", ""),
        }
        result = post("farmerBasicInfo", body, API_FBI)
        farmer_id = result["id"]
        print(f"OK {row['national_id']} → {farmer_id}")

        # Create residency
        residency = {
            "parent_id": farmer_id,
            "district": row["district"],
            "village": row["village"],
            "gpsLatitude": row.get("lat", ""),
            "gpsLongitude": row.get("lon", ""),
        }
        post("farmerResidency", residency, API_RES)

```

The full example script handles errors, logs failures to a CSV, supports resume-on-restart via the existing `national_id` check, and processes parcels in a second pass.

Section 8 — Common gotchas

- **Date format.** Joget DatePickers expect YYYY-MM-DD strings, not ISO datetime. 1985-10-01 is correct; 1985-10-01T00:00:00Z is not.
- **Foreign-key codes are case-sensitive.** District maseru is not the same as Maseru. Always read the master-data list with /list and match exactly.
- **Numeric fields are strings.** Joget stores all form-data as text in `app_fd_*`. Send "3.5" not 3.5 for hectares — both work but the string form is what's actually stored, so consistency matters when you're scripting.
- **Validation errors return HTTP 400 with a JSON body.** Parse it for the field name and the error message. Don't retry blindly — fix the record.
- **Duplicate national_id triggers the validator.** If you re-run a migration after a partial failure, expect HTTP 400 saying "duplicate national_id" for already-imported rows. Skip them — they're already in the system.
- **parent_id is optional but useful.** If you don't set it, the sub-forms still work but the wizard navigation in the UI won't re-bind correctly. Operators see the data when they

click into each sub-form individually but the wizard's Next/Previous flow is broken. For migration data that operators won't edit, this is fine. For migration data they might continue to edit, set `parent_id` properly.

- **The `/list` endpoint is NOT functional.** It's enabled in the API config but `AppFormAPI` doesn't implement listing — calls return `{ }`. Use the static CSVs in `mdm_reference/` (see Section 6) for master-data validation. For “is this NID already in the registry?” questions, the migration script tracks state locally via `migration_log.csv` rather than querying the portal. If you need to bulk-read records during the migration (rare), ask MAFSN ICT to run a `SELECT` for you.
- **The `id` returned in the response is a UUID, not the business code.** Joget's internal `id` and the business code (`code`, `national_id`, etc.) are separate. Use the UUID for `parent_id` / foreign-key fields; the business code is what humans see in the dropdowns and reports.
- **Don't bypass via `formcreator/seed` for transactional data.** That endpoint exists for fixture seeding (master data, demo data) and intentionally bypasses some validators. For citizen-data migration, always use the per-form `/api/form/<id>` endpoints.

Section 9 — Error handling pattern

The form-data APIs use a slightly unusual response shape: **HTTP 200 even when validation fails**, with the failure surfaced inside the JSON body.

Success response (record was saved):

```
{ "id": "abc12345-...",  
  "national_id": "066257236627",  
  "first_name": "Mants'ali",  
  ... (all the fields you sent, echoed back) }
```

The `id` is the saved row's UUID — capture it for any FK references you'll need (e.g. `parent_id` on a sub-form record).

Validation failure (HTTP 200 body, `id` is empty):

```
{ "id": "",  
  "errors": {  
    "national_id": "Missing required value",  
    "first_name": "Missing required value"  
  } }
```

The `errors` object maps each failing field to its error message. To detect a failure, check whether `errors` is present (or whether `id` is empty / missing).

Other failures:

- HTTP 400 with "Bad Request" body → auth / request shape was wrong; request didn't reach the form save path. Check headers + URL.
- HTTP 401 / 403 → credentials rejected.

- HTTP 5xx → server error; retry with backoff.
- HTTP 404 → URL path wrong (most often missing /jw prefix).

Recommended retry / log strategy in your migration script:

Outcome	Action
HTTP 200, id non-empty	Record saved; move to next row
HTTP 200, errors non-empty	Validation failure — log row + the errors map; don't retry
HTTP 400 / 401 / 403	Auth/request misconfigured — STOP the whole run; fix and resume
HTTP 5xx	Server error — exponential backoff, max 3 retries, then log + skip
Network timeout / connection refused	Retry with backoff; if persistent, STOP

After the migration completes, hand the failure log to MAFSN ICT — it's the punchlist for manual cleanup. Common validation failures and how to interpret them:

- "Missing required value" → mandatory field not supplied
- "This value already exists" (on national_id) → duplicate; the row is already in the system. Skip with status already_imported.
- "Numeric value required" → you sent a non-numeric value into a numeric field. Strip whitespace and re-try.

Section 10 — Performance expectations

The system can comfortably handle ~10 records/second per form endpoint. For a 50,000-farmer migration that's about 1.5 hours per form, or roughly 5 hours total if you do basicInfo + residency + income + declaration sequentially.

Parallelisation across form types (basicInfo for farmer N runs simultaneously with residency for farmer N-1) can roughly halve total time. Don't parallelise across the same form (you'll hit lock contention on the underlying Hibernate transactions).

If your pilot has more than 100,000 records, talk to MAFSN ICT before starting — they may want to schedule a quiet window and check that the database has enough free space for the audit log entries that each save generates.

Section 11 — Support

For migration-time questions:

- MAFSN ICT — provides UAT URL, dedicated api_key, support during migration window.
- Portal documentation — _design/ directory in the source repo (architecture, decision log, playbooks).

- For any field not covered here, query the live form definition:

-- read-only; safe

```
SELECT json FROM app_form  
WHERE appid='farmersPortal' AND formid='<form_id>;
```

The elements array in the JSON has every field with its id, className, and any validator configuration.

Authored: 2026-05-06. If your environment differs (UAT, prod), the api_id values in this document should still match — they're stable across environments — but the api_key and base URL will differ. Always confirm with MAFSN ICT before running against a live system.